

Trabajo Práctico 4 - Módulos de Kernel y llamadas a Sistema

Integrantes

- Santiago Alasia
- Lucia Feiguin
- Elena Monutti

Link del Repositorio

```
https://github.com/SantiagoAlasia/The-Pipelineers/tree/main/TP4
```

Introducción

Los módulos del kernel constituyen uno de los mecanismos fundamentales mediante los cuales Linux extiende sus funcionalidades de manera dinámica, permitiendo agregar soporte para dispositivos, sistemas de archivos y distintas características del sistema sin necesidad de recompilar o reiniciar el kernel completo.

En este trabajo práctico se estudió el funcionamiento básico de los módulos del kernel Linux, su compilación, carga y descarga dinámica, así como también distintos conceptos relacionados con la arquitectura del sistema operativo, el espacio de usuario y el espacio del kernel, drivers y llamadas al sistema.

Además, se analizaron mecanismos de seguridad modernos relacionados con Secure Boot y la firma digital de módulos, evaluando su importancia para prevenir la carga de código malicioso dentro del kernel.

Objetivos

- Comprender el funcionamiento básico de los módulos del kernel Linux.
- Compilar, cargar y descargar módulos del kernel utilizando herramientas del sistema.
- Analizar diferencias entre programas de usuario y módulos del kernel.
- Investigar el funcionamiento de drivers y dispositivos en Linux.
- Observar llamadas al sistema mediante herramientas de tracing.
- Comprender el manejo de errores de memoria como segmentation faults.
- Investigar mecanismos de seguridad relacionados con Secure Boot y firmas digitales.
- Realizar la firma digital de un módulo de kernel y verificar su autenticidad.
- Analizar implicancias de seguridad asociadas a la carga de módulos no confiables.

Desafío 1

¿Qué es checkinstall y para qué sirve?

checkinstall es una herramienta que permite crear paquetes instalables (**.deb**, **.rpm**, etc.) a partir del proceso de instalación de un programa compilado desde código fuente.

Normalmente, muchos programas se instalan utilizando:

```
make install
```

Sin embargo, este método copia archivos directamente al sistema sin que el gestor de paquetes tenga registro de ellos. Como consecuencia, posteriormente resulta difícil desinstalar o administrar dichos archivos.

checkinstall soluciona este problema interceptando el proceso de instalación y generando un paquete instalable administrado por el sistema de paquetes de Linux.

Entre sus principales ventajas se encuentran:

- facilidad para desinstalar software
- mejor control de versiones
- integración con el gestor de paquetes del sistema
- posibilidad de distribuir aplicaciones compiladas localmente

¿Se animan a usarlo para empaquetar un hello world?

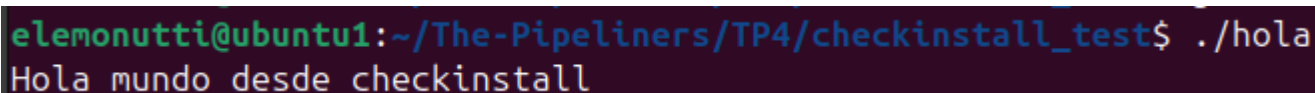
Primero se desarrolló un programa simple en C:

```
#include <stdio.h>

int main() {
    printf("Hola mundo desde checkinstall\n");
    return 0;
}
```

Luego se compiló utilizando:

```
gcc hola.c -o hola
```



```
elemonutti@ubuntu1:~/The-Pipeline/TP4/checkinstall_test$ ./hola
Hola mundo desde checkinstall
```

Figura 1: Ejecución del programa Hello World.

Posteriormente se creó un **Makefile** con una regla **install** para permitir el uso de **checkinstall**.

Finalmente se ejecutó:

```
sudo checkinstall
```

La herramienta generó automáticamente un paquete `.deb` instalable administrado por el sistema de paquetes.

```
*****
Done. The new package has been installed and saved to
/home/elemonutti/The-Pipeline/TP4/checkinstall_test/checkinstall-test_20260521-1_amd64.deb
You can remove it from your system anytime using:
    dpkg -r checkinstall-test
*****
```

Figura 2: Generación del paquete utilizando checkinstall.

```
elemonutti@ubuntu1:~/The-Pipeline/TP4/checkinstall_test$ ls
checkinstall-test_20260521-1_amd64.deb  description-pak  hola  hola.c  Makefile
```

Figura 3: Paquete `.deb` generado.

Posteriormente se instaló el paquete generado utilizando `dpkg`:

```
sudo dpkg -i checkinstall-test_20260521-1_amd64.deb
```

Luego se verificó su correcto funcionamiento ejecutando el binario instalado:

```
hola
```

```
elemonutti@ubuntu1:~/The-Pipeline/TP4/checkinstall_test$ hola
Hola mundo desde checkinstall
```

Figura 4: Instalación y ejecución del paquete generado.

Revisar la bibliografía para impulsar acciones que permitan mejorar la seguridad del kernel, concretamente: evitando cargar módulos que no estén firmados. rootkits?

Los módulos del kernel se ejecutan en espacio privilegiado (*kernel space*), por lo que poseen acceso completo al hardware y a estructuras internas del sistema operativo. Debido a esto, un módulo malicioso puede comprometer completamente la seguridad y estabilidad del sistema.

Una de las principales amenazas relacionadas con módulos del kernel son los **rootkits**, programas maliciosos diseñados para ocultar procesos, archivos, conexiones de red o elevar privilegios dentro del sistema.

Para reducir estos riesgos, Linux incorpora mecanismos de verificación mediante **firmas digitales**. Cuando Secure Boot se encuentra habilitado, el kernel puede configurarse para permitir únicamente la carga de módulos firmados por entidades confiables.

Esto garantiza:

- autenticidad del módulo
- integridad del código
- protección contra modificaciones maliciosas
- prevención de rootkits a nivel kernel

En caso de que un módulo no esté firmado o utilice una clave no confiable, el sistema puede impedir su carga.

Desafío 2

Debe tener respuestas precisas a las siguientes preguntas y sentencias:

1. ¿Qué funciones tiene disponible un programa y un módulo?

Un programa de usuario utiliza funciones provistas por bibliotecas del sistema (como libc) y accede a los recursos del sistema operativo mediante **system calls (syscalls)**. Algunas de las más comunes son:

```
open(),  
read(),  
write(),  
etc.
```

Estas llamadas permiten interactuar con archivos, dispositivos y otros recursos del sistema de **manera controlada y segura**, ya que el sistema operativo restringe el acceso directo al hardware y a la memoria del kernel.

En cambio, un módulo del kernel se ejecuta en espacio de kernel, por lo que no utiliza bibliotecas de usuario como libc. Las funciones y símbolos que puede utilizar son proporcionados por el kernel.

Estas funciones pueden observarse en el archivo `/proc/kallsyms`, el cual contiene la tabla de símbolos del kernel cargados actualmente en memoria, incluyendo funciones y variables globales disponibles para otros módulos.

Por ejemplo:

```
printk()
```

2. Espacio de usuario o espacio del kernel.

Cada proceso de usuario posee su propio espacio de memoria aislado, lo que evita que un programa pueda acceder directamente a la memoria de otro proceso o del kernel. Esta separación aporta estabilidad y seguridad al sistema. Cuando un programa de usuario falla, normalmente solo finaliza dicho proceso sin afectar al resto del sistema operativo.

Por otro lado, el kernel y los módulos cargados se ejecutan dentro del espacio del kernel, compartiendo el mismo **espacio de memoria privilegiado**. Debido a esto, un error dentro de un módulo del kernel puede

comprometer la estabilidad de todo el sistema, provocando fallas graves.

En la *Figura 1* puede observarse una distribución lógica entre el espacio de usuario y el espacio del kernel.

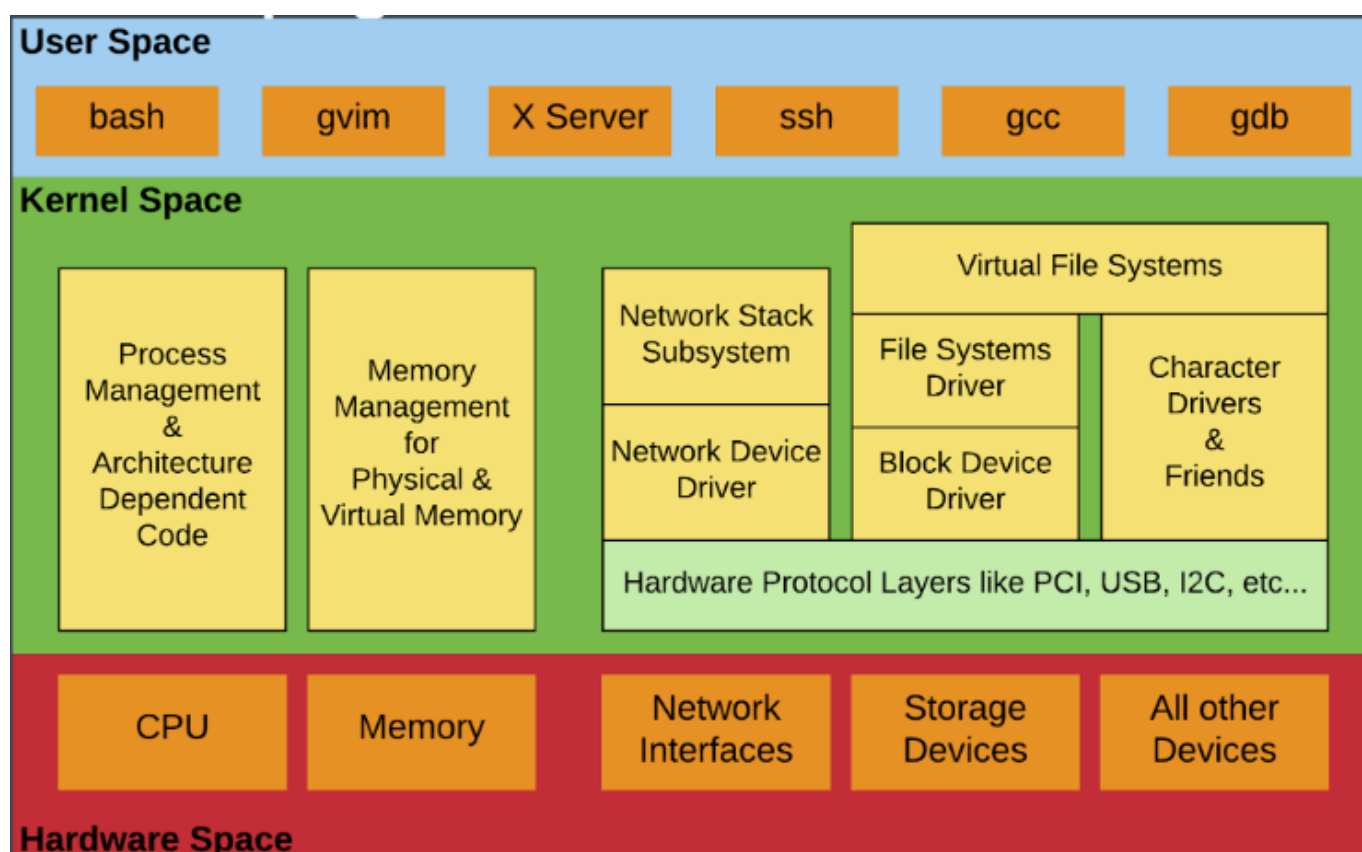


Figura 5: Espacio de Usuario vs Espacio del Kernel

3. Espacio de datos.

El espacio de datos es la región de memoria de un programa donde se almacenan las **variables** y **datos utilizados** durante su ejecución. Este es fundamental porque contiene el estado y la información que utiliza el programa mientras se ejecuta.

Dentro de la memoria de un proceso existen distintas secciones, cada una con un propósito específico. Las secciones `.data` y `.bss` forman parte del espacio de datos del programa. Además, durante la ejecución el programa puede reservar memoria dinámica en el **heap** utilizando funciones como `malloc()`.

En el caso de los módulos del kernel sucede algo similar, aunque la memoria es administrada directamente por el kernel mediante funciones como `kmalloc()`.

4. Drivers. Investigar contenido de `/dev`.

Los **Drivers** o controladores de dispositivos son módulos especiales que proporcionan funcionalidad para un hardware específico, como discos, teclados, placas de red, GPU o dispositivos USB. Como en los sistemas **Unix** los dispositivos se mapean como archivos, los vamos a encontrar asociados con alguna entrada en `/dev`. Esto permite interactuar con el hardware utilizando operaciones similares a las realizadas sobre archivos comunes, como `read()` o `write()`.

Para inspeccionar el contenido de `/dev` pueden utilizarse los siguientes comandos:

```
cd /dev
```

```
ls -l
```

La salida mostrará distintos archivos asociados a dispositivos del sistema. Por ejemplo:

```
brw-rw----  1 root disk      8,      0 jul 11  2025 sda
crw-rw-rw-  1 root tty       5,      0 may 20 12:15 tty
```

Obs: En esta salida puede observarse que al comienzo aparece una letra que indica el tipo de dispositivo:

- **c** dispositivo de caracteres (character device)
- **b** dispositivo de bloques (block device)

Los dispositivos de caracteres transfieren información byte a byte, como terminales o teclados. En cambio, los dispositivos de bloques trabajan con bloques de datos, como discos rígidos o pendrives.

Además, aparecen dos números llamados **mayor** y **menor**:

- El número **mayor** identifica al driver o controlador asociado al dispositivo.
- El número **menor** identifica una instancia específica controlada por ese driver.

Por ejemplo, distintos discos pueden compartir el mismo número mayor (mismo driver) pero tener distintos menor para diferenciar cada dispositivo físico o partición.

Desafío 3

Pasos para la compilación y carga de un módulo de kernel.

```
cd kernel-modules/part1/module
```

```
make clean
make all
```

```
sudo insmod mimodulo.ko
```

```
sudo dmesg
```

```
[81586.712046] wlp2s0: RX AssocResp from 00:21:29:63:51:34 (capab=0x411 status=0 aid=4)
[81586.714876] wlp2s0: associated
[84754.004618] Modulo cargado en el kernel.
~/Documentos/Proyectos-UNC/The-Pipelineers/TP4/kernel-modules/part1/module > █
```

Figura 6: Log de eventos luego de cargar el módulo de kernel.

```
lsmod | grep mod
```

```
~/Documentos/Proyectos-UNC/The-Pipelineers/TP4/kernel-modules/part1/module > lsmod | grep mod
mimodulo                12288  0
nvidia_modeset          1638400  6 nvidia_drm
nvidia                  104071168  167 nvidia_uvm,nvidia_modeset
video                   77824  4 asus_wmi,asus_nb_wmi,i915,nvidia_modeset
```

Figura 7: Lista de módulos cargados filtrados por "mod".

```
sudo rmmod mimodulo
```

```
sudo dmesg
```

```
[84754.004618] Modulo cargado en el kernel.
[84993.493476] Modulo descargado del kernel.
~/Documentos/Proyectos-UNC/The-Pipelineers/TP4/kernel-modules/part1/module > █
```

Figura 8: Log de eventos luego de quitar el módulo de kernel.

```
lsmod | grep mod
```

```
~/Documentos/Proyectos-UNC/The-Pipelineers/TP4/kernel-modules/part1/module > lsmod | grep mod
nvidia_modeset          1638400  6 nvidia_drm
nvidia                  104071168  167 nvidia_uvm,nvidia_modeset
video                   77824  4 asus_wmi,asus_nb_wmi,i915,nvidia_modeset
```

Figura 9: Lista de módulos cargados filtrados por "mod".

/proc/modules es un archivo del virtual filesystem que muestra los módulos del kernel que están actualmente cargados en memoria. Para poder ver el contenido podemos ejecutar el siguiente comando.

```
cat /proc/modules | grep mod
```

```
~/Documentos/Proyectos-UNC/The-Pipeline/TP4/kernel-modules/part1/module > sudo insmod mimodulo.ko
~/Documentos/Proyectos-UNC/The-Pipeline/TP4/kernel-modules/part1/module > cat /proc/modules | grep mod
mimodulo 12288 0 - Live 0x0000000000000000 (OE)
```

Figura 10: Lista de módulos cargados filtrados por "mod".

```
modinfo mimodulo.ko
```

```
~/Documentos/Proyectos-UNC/The-Pipeline/TP4/kernel-modules/part1/module > modinfo mimodulo.ko
filename:      /home/santiagoolasia/Documentos/Proyectos-UNC/The-Pipeline/TP4/kernel-modules/part1/module/mimodulo.ko
author:        Catedra de SdeC
description:    Primer modulo ejemplo
license:        GPL
srcversion:     C6390D617B2101FB1B600A9
depends:
retpoline:     Y
name:          mimodulo
vermagic:       6.8.0-87-generic SMP preempt mod_unload modversions
```

Figura 11: Descripción del modulo.

```
modinfo /lib/modules/$(uname -r)/kernel/crypto/des_generic.ko
```

```
santiagoolasia@santiagoolasia-X510UQ/lib/modules/6.8.0-87-generic/kernel/crypto
~/lib/modules/6.8.0-87-generic/kernel/crypto > modinfo des_generic.ko.zst
filename:      /lib/modules/6.8.0-87-generic/kernel/crypto/des_generic.ko.zst
alias:         crypto-des3_edc-generic
alias:         des3_edc-generic
alias:         crypto-des3_edc
alias:         des3_edc
alias:         crypto-des-generic
alias:         des-generic
alias:         crypto-des
alias:         des
author:         Dag Arne Osvik <da@osvik.no>
description:    DES & Triple DES EDE Cipher Algorithms
license:        GPL
srcversion:     B56606AD918CF0074D320DB
depends:         libdes
retpoline:     Y
intree:         Y
name:          des_generic
vermagic:       6.8.0-87-generic SMP preempt mod_unload modversions
sig_id:         PKCS#7
signer:         Build time autogenerated kernel key
sig key:        56:94:49:42:D9:BD:8C:0E:DE:5E:D6:00:A2:9C:DE:1E:22:DD:6D:67
sig hash algo:  sha512
signature:      21:44:7F:A4:54:DA:57:BC:04:2A:A1:F9:11:73:16:A8:66:20:C5:D8:
88:F9:A7:21:42:04:FC:6E:FB:BD:A5:3B:C4:07:6C:CA:43:00:A4:13:
4F:D5:05:18:F2:EE:98:12:BB:F4:0F:A5:A3:A0:0E:15:5E:33:92:0E:
25:85:F1:A9:72:8C:88:CD:68:47:C7:C2:4F:AB:30:14:7B:4E:B0:28:
58:C7:36:6E:71:95:28:28:FB:8E:B2:1D:7A:6D:74:A8:5F:F7:C2:78:
16:3C:06:7E:9A:41:3D:FA:12:48:7D:43:F9:C7:DF:89:D8:3D:C1:77:
11:1F:BC:E9:18:43:AC:97:2B:0A:BF:84:68:C9:70:88:02:4D:58:29:
E7:BA:88:BC:D8:AB:EB:4F:9C:2C:66:59:E7:F3:62:46:77:B6:C7:5D:
EE:2B:E6:DE:E4:96:25:C8:6B:8F:9E:EA:C3:8D:CF:0C:80:72:0A:EF:
8E:C7:D4:AC:02:69:59:15:C1:25:CF:91:01:A0:89:CA:85:01:A6:96:
72:F6:9F:D3:DC:3E:0C:18:97:99:CB:CC:DF:3B:33:B7:A5:8B:97:85:
71:65:4D:C9:AB:43:F2:4E:FA:15:5D:96:3C:41:AD:C3:92:3D:5C:1B:
37:4C:50:FB:ED:79:C3:BC:5C:77:EA:22:14:FC:C8:4E:34:25:D4:94:
E5:3E:97:24:B9:C2:D6:88:99:08:DE:CA:F0:01:02:C5:D7:D5:AB:0B:
1B:96:05:2E:09:7D:00:75:8B:BB:D9:0A:0B:04:27:4D:D4:02:E3:46:
7E:CE:F3:1A:1B:78:46:24:E1:29:D7:15:59:9C:45:CA:D9:B1:A2:94:
BA:C5:CB:6A:1F:B1:02:05:51:A9:6B:D5:45:06:4B:73:E6:80:C6:C4:
68:16:6A:FE:20:86:61:2E:5C:18:7E:C1:98:89:34:77:62:CB:EA:B2:
E3:16:27:0C:E4:51:1F:D0:19:4A:61:E1:43:13:09:51:58:02:F7:23:
EE:42:72:59:38:93:FB:EB:11:F9:62:C0:3C:41:01:56:05:13:D7:57:
EF:7C:57:83:92:FE:CB:9F:90:CA:E8:34:20:6C:31:ED:DF:F0:15:E1:
5C:B7:A7:0B:8D:1E:A0:2F:EE:48:47:33:6B:81:24:76:90:BA:4D:BD:
66:DE:CF:02:48:05:B1:D8:77:06:05:0F:FB:BD:71:57:25:40:2F:26:
2B:F5:71:FC:4A:09:7E:48:74:97:80:0E:7B:57:7D:BE:AB:02:CD:7E:
07:90:15:D8:83:8E:62:08:6D:07:5C:AE:CC:08:35:F7:E2:07:AE:29:
46:79:31:7E:ED:B4:84:D5:47:0D:A2:3F
```

Figura 12: Descripción del modulo crypto.

Preguntas

1. ¿Qué diferencias se pueden observar entre los dos modinfo?

Observando las **Figuras 7 y 8** puede verse que el módulo desarrollado manualmente (**mimodulo.ko**) posee una cantidad reducida de metadata, mientras que el módulo oficial del kernel incluye información adicional como:

- alias
- dependencias
- versión
- firma digital

Uno de los campos más importantes es el correspondiente a la **firma digital**, ya que permite verificar la autenticidad e integridad del módulo antes de que sea cargado por el kernel. Esto resulta especialmente importante en sistemas con Secure Boot habilitado, donde únicamente pueden cargarse módulos firmados por claves consideradas confiables por el sistema.

2. ¿Qué drivers/modulos estan cargados en sus propias pc? comparar las salidas con las computadoras de cada integrante del grupo. Expliquen las diferencias. **Carguen un txt con la salida de cada integrante en el repo y pongan un diff en el informe.**

Para revisar los modulos que estan cargados y generar un .txt podemos ejecutar los siguientes comandos:

```
cd drivers  
  
lsmod > <apellido_del_interante>.txt
```

Para comparar los módulos cargados en las computadoras de los integrantes del grupo, cada uno generó un archivo de texto con la salida del comando `lsmod`.

Los archivos agregados al repositorio fueron:

- `drivers/aliasia.txt`
- `drivers/monutti.txt`
- `drivers/feiguin.txt`

Luego se generaron los archivos de diferencias utilizando el comando `diff -u`:

```
diff -u feiguin.txt monutti.txt > diff_feiguin_monutti.txt  
diff -u feiguin.txt aliasia.txt > diff_feiguin_aliasia.txt
```

Los archivos generados fueron:

```
drivers/diff_feiguin_monutti.txt  
drivers/diff_feiguin_aliasia.txt
```

A partir de la comparación se observaron diferencias entre los módulos cargados en cada sistema. Estas diferencias se deben principalmente al hardware disponible, los dispositivos conectados, los servicios activos y el entorno donde se ejecuta cada sistema operativo.

En la salida correspondiente a Lucia Feiguin se observan módulos como `vboxsf`, `vboxguest` y `vboxvideo`, los cuales están asociados a VirtualBox. Esto indica que el sistema Linux utilizado se está ejecutando dentro de

un entorno virtualizado. Por este motivo, aparecen módulos específicos de virtualización que pueden no estar presentes en las computadoras de los otros integrantes.

También pueden existir diferencias en módulos relacionados con red, sonido, Bluetooth, almacenamiento, GPU, dispositivos USB o herramientas del sistema. Esto demuestra que Linux carga dinámicamente distintos drivers según las características particulares de cada equipo y los recursos que se estén utilizando al momento de ejecutar `lsmod`.

3. ¿Cuales no están cargados pero están disponibles? ¿Que pasa cuando el driver de un dispositivo no está disponible?.

Para determinar qué módulos se encuentran disponibles en el sistema, pero no están actualmente cargados en el kernel, pueden utilizarse distintos comandos.

En primer lugar, podemos listar todos los módulos disponibles:

```
find /lib/modules/$(uname -r) -name "*.ko.zst"
```

Este comando muestra los módulos presentes en el **file system**, es decir, aquellos que el kernel puede cargar si son necesarios.

Luego, puede filtrarse la búsqueda según el módulo que se desea inspeccionar:

```
find /lib/modules/$(uname -r) | grep <nombre_del_modulo>
```

Si el módulo aparece en la salida, significa que está disponible en el sistema.

Finalmente, para verificar si dicho módulo se encuentra actualmente **cargado** en memoria, puede utilizarse:

```
lsmod | grep <nombre_del_modulo>
```

Si el comando no devuelve resultados, entonces el módulo está **disponible pero no cargado**.

Por otro lado, si el driver de un dispositivo no se encuentra disponible, el sistema operativo no podrá comunicarse correctamente con dicho hardware. Como consecuencia, el dispositivo puede no ser detectado, funcionar de manera limitada o directamente quedar inutilizable dentro del sistema operativo.

4. Correr `hwinfo` en una pc real con hw real y agregar la url de la información de hw en el reporte.

`hwinfo` es una herramienta de Linux que permite obtener información detallada del hardware del sistema.

```
hwinfo > hwinfo.txt
```

La salida la podemos encontrar en el archivo `hwinfo.txt` del repo.

5. ¿Qué diferencia existe entre un módulo y un programa?

Existen varias diferencias entre un **programa de usuario** y un **módulo del kernel**, una de las más importantes es la forma en la que comienzan su ejecución y cómo interactúan con el sistema operativo.

Un programa inicia su ejecución en la función **main()**, ejecuta sus instrucciones y, al finalizar, retorna el control al sistema operativo o al proceso que lo invocó.

En cambio, un módulo del kernel posee **funciones especiales** de inicialización y finalización (`module_init()`, `module_exit()`).

La función de inicialización se ejecuta cuando el módulo es **cargado en el kernel** y se utiliza para registrar la funcionalidad que el módulo provee, inicializar estructuras o reservar recursos necesarios. Posteriormente, el módulo permanece cargado y el kernel puede utilizar sus servicios cuando sean requeridos. La función de salida se ejecuta al **descargar** el módulo y permite liberar recursos y limpiar el estado interno.

Otra diferencia fundamental es el **entorno de ejecución**. Los programas se ejecutan en el **espacio de usuario (user space)**, donde poseen acceso restringido a los recursos del sistema. No pueden acceder directamente al hardware ni a la memoria del kernel. Para realizar operaciones privilegiadas, como acceder a archivos, dispositivos o memoria protegida, deben solicitar **servicios al sistema operativo** mediante system calls (syscalls).

Por el contrario, los módulos se ejecutan dentro del **espacio del kernel (kernel space)**, compartiendo el mismo entorno de ejecución que el kernel Linux. Debido a esto, poseen acceso privilegiado al hardware, memoria física y estructuras internas del sistema operativo.

Como consecuencia, si un programa de usuario falla, normalmente solo termina el proceso asociado. Sin embargo, si un módulo del kernel contiene errores, puede comprometer la estabilidad completa del sistema.

Otra característica importante es que los módulos pueden cargarse y descargarse dinámicamente en tiempo de ejecución permitiendo extender las funcionalidades del kernel sin necesidad de reiniciar el sistema operativo.

6. ¿Cómo puede ver una lista de las llamadas al sistema que realiza un simple helloworld en c?

Para observar las system calls que realiza un programa en Linux puede utilizarse la herramienta strace.

Primero creamos un programa simple:

```
#include <stdio.h>

int main() {
    printf("Hola mundo\n");
    return 0;
}
```

Compilación:

```
gcc -Wall hola.c -o hola
```

Luego ejecutamos strace:

```
strace ./hola
```

Este comando muestra todas las llamadas al sistema realizadas por el programa, incluyendo operaciones de carga de librerías dinámicas, acceso a memoria, escritura en pantalla y finalización del proceso.

También pueden utilizarse variantes útiles:

```
strace -tt ./hola
```

Muestra timestamps precisos para cada syscall.

```
strace -c ./hola
```

Genera un resumen estadístico de las llamadas al sistema realizadas.

```

elemnutti@ubuntu1:~/The-Pipeline/TP4/strace_test$ strace -c ./hola
Hola mundo

```

% time	seconds	usecs/call	calls	errors	syscall
44.74	0.000421	421	1		execve
18.81	0.000177	22	8		mmap
5.42	0.000051	17	3		mprotect
4.68	0.000044	22	2		openat
3.40	0.000032	10	3		fstat
3.19	0.000030	10	3		brk
2.98	0.000028	28	1		munmap
2.44	0.000023	11	2		close
2.34	0.000022	22	1	1	access
2.13	0.000020	10	2		pread64
2.02	0.000019	19	1		write
1.38	0.000013	13	1		getrandom
1.28	0.000012	12	1		arch_prctl
1.17	0.000011	11	1		read
1.06	0.000010	10	1		set_tid_address
1.06	0.000010	10	1		rseq
0.96	0.000009	9	1		set_robust_list
0.96	0.000009	9	1		prlimit64
100.00	0.000941	27	34	1	total

Figura 13: Resumen de system calls obtenidas mediante strace.

Entre las syscalls más comunes que aparecerán se encuentran:

- `execve()`
- `mmap()`
- `openat()`
- `read()`
- `write()`
- `close()`

Por ejemplo, la llamada:

```
write(1, "Hola mundo\n", 11)
```

indica que el programa escribe el texto en el descriptor de archivo 1, correspondiente a la salida estándar (`stdout`).

Esto demuestra que incluso un programa muy simple depende de múltiples servicios proporcionados por el kernel Linux.

7. ¿Qué es un segmentation fault? ¿Cómo lo maneja el kernel y como lo hace un programa?

Un segmentation fault (o simplemente segfault) ocurre cuando un programa intenta acceder a una región de memoria que no tiene permitida.

Algunos ejemplos comunes son:

- acceder a un puntero NULL
- escribir fuera de los límites de un arreglo
- acceder a memoria liberada
- intentar ejecutar memoria no ejecutable

Ejemplo:

```
int *ptr = NULL;
*ptr = 10;
```

En este caso el proceso intenta escribir en una dirección inválida y el procesador genera una excepción de memoria.

La MMU (Memory Management Unit) detecta el acceso inválido y genera una excepción que luego es manejada por el kernel Linux. Cuando ocurre el acceso inválido, el hardware genera una interrupción conocida como page fault y el kernel verifica si el acceso es válido.

Si el acceso no está permitido, el kernel envía al proceso la señal:

SIGSEGV

Por defecto, esta señal finaliza el programa y puede generar un core dump para depuración.

En programas de usuario, normalmente el fallo solo afecta al proceso que cometió el error gracias al aislamiento de memoria entre procesos.

Sin embargo, en el caso de un módulo del kernel, el código se ejecuta en espacio privilegiado compartiendo memoria con el kernel. Por esta razón, un acceso inválido dentro de un módulo puede provocar:

- kernel panic
- congelamiento del sistema
- reinicio completo
- corrupción de memoria

Esto demuestra por qué el desarrollo de módulos del kernel requiere mucho más cuidado que la programación tradicional en espacio de usuario.

8. ¿Se animan a intentar firmar un módulo de kernel ? y documentar el proceso ?

<https://askubuntu.com/questions/770205/how-to-sign-kernel-modules-with-sign-file>

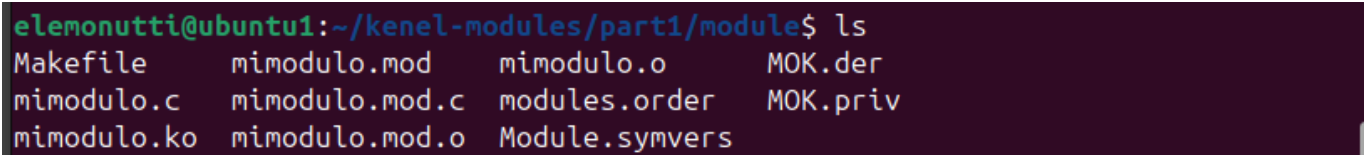
Con el objetivo de mejorar la seguridad del sistema y verificar la autenticidad de los módulos cargados en el kernel, se realizó la firma digital del módulo `mimodulo.ko`.

Primero se generó una clave privada junto con un certificado X.509 utilizando OpenSSL:

```
openssl req -new -x509 -newkey rsa:2048 -keyout MOK.priv -outform DER -out  
MOK.der -nodes -days 36500 -subj "/CN=ModuloKernel/"
```

Esto generó los archivos:

- MOK.priv
- MOK.der



```
elemnutt@ubuntu1:~/kernel-modules/part1/module$ ls  
Makefile      mimodulo.mod  mimodulo.o    MOK.der  
mimodulo.c    mimodulo.mod.c  modules.order  MOK.priv  
mimodulo.ko   mimodulo.mod.o  Module.symvers
```

Figura 14: Generación de claves y certificado para la firma del módulo.

Posteriormente se utilizó el script `sign-file`, provisto por los headers del kernel Linux, para aplicar la firma digital al módulo:

```
/usr/src/linux-headers-$(uname -r)/scripts/sign-file sha256 MOK.priv  
MOK.der mimodulo.ko
```

Finalmente se verificó la presencia de la firma utilizando:

```
modinfo mimodulo.ko
```

```

elemonutti@ubuntu1:~/kenel-modules/part1/module$ modinfo mimodulo.ko
filename:          /home/elemonutti/kenel-modules/part1/module/mimodulo.ko
author:            Catedra de SdeC
description:       Primer modulo ejemplo
license:           GPL
srcversion:        C6390D617B2101FB1B600A9
depends:
name:              mimodulo
retpoline:         Y
vermagic:          6.17.0-29-generic SMP preempt mod_unload modversions
sig_id:            PKCS#7
signer:            ModuloKernel
sig_key:           1F:A7:68:0F:4E:90:06:37:E8:7F:B9:DB:A4:00:08:F1:A6:6A:C0:AB
sig_hashalgo:      sha256
signature:         A1:16:21:4F:A5:CE:2B:7C:8C:13:85:FC:37:97:12:27:0B:0A:C7:09:
                    5D:A6:D1:DA:4B:AD:C8:C6:58:58:9D:16:BA:88:9A:75:6C:5E:6E:34:
                    12:00:28:77:F1:23:41:06:CC:F6:8B:30:34:1F:40:4F:47:30:C2:B6:
                    B3:77:75:98:06:54:59:54:39:AA:0D:56:43:D0:EA:38:2D:0D:4A:52:
                    81:7B:97:25:26:4D:CA:6F:ED:8A:88:A2:E6:51:01:B3:3D:8B:74:4F:
                    4D:0D:7D:C8:DA:CE:43:CC:2F:8B:C1:15:40:C1:27:AC:21:25:6D:DC:
                    61:48:10:53:EC:CD:88:F9:AF:40:0C:E7:A9:69:95:43:5D:82:4F:05:
                    82:AE:D3:7F:84:30:AF:88:DD:69:69:E9:44:B8:B9:55:2C:06:28:76:
                    95:9F:05:1D:73:40:1D:01:C3:24:24:94:60:5A:31:3F:01:AB:3C:E3:
                    76:15:46:96:4D:57:50:79:D1:CA:AD:6D:96:F8:6D:D8:B9:CF:1D:5B:
                    A7:7E:D6:4E:16:65:53:03:45:EC:BB:BE:1D:25:01:2E:C8:48:39:91:
                    89:A8:79:8E:95:C7:55:03:C2:EC:C0:28:18:FC:D9:46:50:42:CA:2A:
                    76:FC:D8:47:F2:1F:69:B1:AD:F6:55:32:E5:FD:34:93

```

Figura 15: Verificación de la firma digital del módulo mediante modinfo.

Puede observarse la aparición de campos relacionados con la firma digital, como **signer**, **sig_key** y **signature**, los cuales indican que el módulo fue firmado correctamente.

9. Agregar evidencia de la compilación, carga y descarga de su propio módulo imprimiendo el nombre del equipo en los registros del kernel.

Se modificó el módulo del kernel para imprimir el hostname del sistema en los registros del kernel utilizando la estructura **init_uts_ns**.

Para ello se agregó el siguiente include:

```
#include <linux/utsname.h>
```

Luego se modificó la llamada a **printk()**:

```
printk(KERN_INFO "Modulo cargado en el equipo: %s\n",
init_uts_ns.name.nodename);
```

Posteriormente se recompiló el módulo:

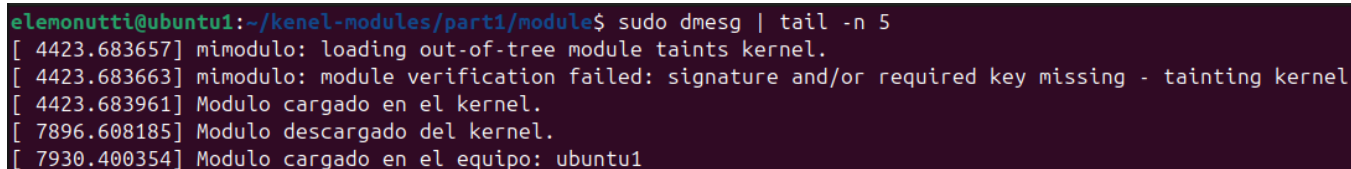

```
make clean
make
```

y se cargó nuevamente utilizando:

```
sudo insmod mimodulo.ko
```

Finalmente se verificó el mensaje generado mediante:

```
sudo dmesg | tail
```



```
elemonutti@ubuntu1:~/kenel-modules/part1/module$ sudo dmesg | tail -n 5
[ 4423.683657] mimodulo: loading out-of-tree module taints kernel.
[ 4423.683663] mimodulo: module verification failed: signature and/or required key missing - tainting kernel
[ 4423.683961] Modulo cargado en el kernel.
[ 7896.608185] Modulo descargado del kernel.
[ 7930.400354] Modulo cargado en el equipo: ubuntu1
```

Figura 16: Registro del kernel mostrando el hostname del equipo.

Puede observarse que el módulo imprime correctamente el nombre del equipo dentro de los logs del kernel.

10. ¿Que pasa si mi compañero con secure boot habilitado intenta cargar un módulo firmado por mí?

Aunque el módulo esté firmado correctamente, el sistema solamente permitirá cargarlo si la clave utilizada para firmarlo pertenece a una entidad confiable para el sistema.

Si el compañero no tiene registrada la clave pública (MOK.der) dentro de su sistema, el kernel rechazará el módulo.

Generalmente aparecerá un mensaje similar a:

Required key not available

o

module verification failed

Esto sucede porque Secure Boot verifica que los módulos hayan sido firmados por claves autorizadas.

Para que el módulo pueda cargarse correctamente, el compañero debe importar previamente el certificado público mediante:

```
mokutil --import MOK.der
```

11. Dada la siguiente nota <https://arstechnica.com/security/2024/08/a-patch-microsoft-spent-2-years-preparing-is-making-a-mess-for-some-linux-users/>

Según el artículo publicado por Ars Technica, Microsoft distribuyó una actualización relacionada con Secure Boot y la vulnerabilidad conocida como BootHole.

El problema principal fue que algunos sistemas Linux con arranque dual comenzaron a tener problemas para iniciar correctamente luego de aplicar las nuevas políticas de seguridad.

12. ¿Cuál fue la consecuencia principal del parche de Microsoft sobre GRUB en sistemas con arranque dual (Linux y Windows)?

La consecuencia principal fue que muchos sistemas Linux dejaron de arrancar correctamente porque las nuevas políticas de Secure Boot bloquearon versiones antiguas de GRUB consideradas vulnerables.

Como resultado:

- algunos equipos no podían iniciar Linux
- aparecían errores relacionados con Secure Boot
- ciertos sistemas entraban directamente al firmware UEFI

Esto afectó especialmente a usuarios con configuraciones dual boot entre Linux y Windows.

13. ¿Qué implicancia tiene desactivar Secure Boot como solución al problema descrito en el artículo?

Desactivar Secure Boot permite volver a cargar bootloaders o módulos no firmados, solucionando temporalmente el problema de compatibilidad.

Sin embargo, esto reduce significativamente la seguridad del sistema porque elimina la validación criptográfica durante el proceso de arranque.

Como consecuencia:

- podrían cargarse bootloaders modificados
- podrían ejecutarse rootkits de bajo nivel
- el sistema queda más expuesto a malware persistente

Por esta razón, desactivar Secure Boot debe considerarse solamente una solución temporal o de diagnóstico.

14. ¿Cuál es el propósito principal del Secure Boot en el proceso de arranque de un sistema?

El propósito principal de Secure Boot es garantizar que únicamente se ejecute software confiable durante el arranque del sistema.

Para ello, Secure Boot verifica firmas digitales de:

- bootloaders
- kernels
- drivers
- módulos

De esta manera se evita que software malicioso pueda ejecutarse antes de que el sistema operativo inicie completamente.

Esto ayuda a proteger el sistema contra amenazas como:

- bootkits
- rootkits
- malware persistente de bajo nivel

Conclusión general

A lo largo del desarrollo del trabajo práctico se logró comprender el funcionamiento general de los módulos del kernel Linux y la diferencia entre el espacio de usuario y el espacio privilegiado del sistema operativo.

La práctica permitió familiarizarse con herramientas fundamentales del entorno Linux, como `lsmod`, `modinfo`, `dmesg`, `strace`, `hwinfo` y `checkinstall`, además de observar cómo interactúan los programas y módulos con el kernel mediante system calls y drivers.

También se analizaron aspectos importantes relacionados con la seguridad del sistema, particularmente el uso de firmas digitales y Secure Boot para evitar la carga de módulos no confiables o potencialmente maliciosos.

Finalmente, la experiencia permitió comprender los riesgos asociados al desarrollo en espacio de kernel, donde errores simples pueden comprometer la estabilidad completa del sistema operativo.